



MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana



DATA STRUCTURES

QUESTION BANK





MODULE – I

S.No	Short Answer Questions	Page no
1	Define Data Structure.	6
2	Define Recursion.	6
3	Why are Data Structures needed?	6
4	What is the difference between Time Complexity and Space Complexity?	6
5	What is Algorithm Efficiency?	6
6	What is Big-O notation?	6
7	Define Multidimensional Array.	6

S.No	Long Answer Questions	Page no
1	Explain the Tower of Hanoi problem in detail. Write the recursive program for solving it and describe the steps involved in moving the disks from source to destination.	7
2	a) Explain the advanced array processing techniques such as Prefix Sum, Sliding Window. Discuss their significance and applications in solving computational problems. b) Apply the Sliding Window technique to find the maximum sum of a subarray of size $k = 3$ for $A = [2, 1, 5, 1, 3, 2]$.	8
3	Explain the concept of Asymptotic analysis in algorithm design. Describe the different types of asymptotic notations such as Big-O, Omega, and Theta, and explain how they are used to evaluate the efficiency of algorithms with suitable examples.	10
4	Construct Prefix Sum for $A = [2, 4, 6, 8, 10]$ and find sum from index 1 to 3	11
5	a) Define Recursion? Explain different types of recursion. illustrate the concept with an example b) write a program to find factorial using recursion.	12
6	Write a program to perform Linear Search on an array.	14



MODULE – II

S.No	Short Answer Questions	Page no
1	What is BubbleSort time complexity in the worst case?	15
2	Define Sorting. List few sorting Techniques.	15
3	Define Hashing.	15
4	Define Binary Search and Mention the Time complexity of Binary Search?	15
5	Define Merge Sort and mention its time complexity?	15
6	What is a Hash Table? List few hash functions	16

S.No	Long Answer Questions	Page no
1	Write a Program to perform sorting using Bubble Sort.	16
2	Discuss the Bubble Sort algorithm and explain how elements are compared and swapped during the sorting process. Illustrate the working of the algorithm with a suitable example.	17
3	Write a Program to perform Binary Search on an Array.	19
4	Apply Selection Sort to sort the array $A = [85, 60, 25, 10, 40, 15]$.	20
5	Explain the Insertion Sort technique and Describe how elements are inserted into their correct positions in the sorted list. Provide an example and write its time complexity	21
6	Describe the Merge Sort algorithm based on the divide-and-conquer approach. Explain the steps involved in dividing and merging the list along with its complexity analysis.	22



MODULE – III

S.No	Short Answer Questions	Page no
1	Define Linked List and how is it different from an array.	26
2	Define Singly Linked List.	26
3	What is a Node in a Linked List?	26
4	Define a Doubly Linked List.	26
5	Define String in data structures.	27
6	What is String Manipulation?	27

S.No	Long Answer Questions	Page no
1	Discuss the process of traversing and searching elements in a linked list. Explain how nodes are accessed sequentially and provide a suitable example.	27
2	Explain the concept of linked lists as a dynamic data structure. Describe how a linked list is represented in memory and illustrate its structure with a diagram.	28
3	Describe the insertion operation in a linked list at different positions such as beginning, middle and end. Explain the procedure with examples.	30
4	Explain the deletion operation in a linked list. Discuss the steps involved in deleting nodes from different positions in the list.	32
5	Discuss the concept of strings in data structures. Explain various string manipulation techniques used in programming.	34
6	Explain the structure and advantages of a Doubly Linked List.	36



MODULE – IV

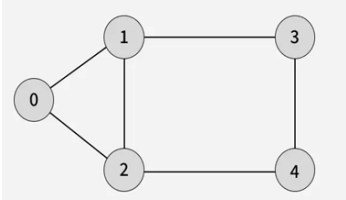
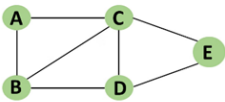
S.No	Short Answer Questions	Page no
1	What is a Stack?	37
2	What are the operations performed on Stack.	37
3	What is a Monotonic Stack ? What is Next Greater Element problem?	37
4	Define Infix, Prefix and Postfix expressions.	38
5	Define Queue and its operations	38
6	What is a Priority Queue?	39

S.No	Long Answer Questions	Page no
1	Explain the concept of stack data structure and describe its basic operations such as push and pop with suitable example.	39
2	Write a program to implement stack operations.	41
3	Convert the following expressions into in to postfix form using stack. a) $A-B/C*D+E$ b) $(A+B)*C+(D-E)/F$	42
4	Explain the various applications of stack in computer science.	43
5	Explain the concept of Queue data structure and describe its operations such as enqueue and dequeue with suitable example	45
6	Write a program to implement Queue operations,	47



MODULE – V

S.No	Short Answer Questions	Page no
1	Define Binary Tree.	48
2	What is a Binary Search Tree?	48
3	What is a Heap in Data Structures?	48
4	What is Breadth First Search?	49
5	What is Depth First Search?	49
6	Define Graph.	49

S.No	Long Answer Questions	Page no
1	Construct a Binary Search Tree for the elements 45, 15, 79, 90, 10, 55, 12, 20, 50.	49
2	Write a Python program to create a Max Heap from a given list of elements and display the heap.	53
3	Explain the concept of Heap in data structures.	54
4	Build a Max Heap from the elements 45, 36, 54, 27, 63, 72, 61, 18.	56
5	Consider the graph given below. Perform Breadth First Search (BFS) traversal starting from vertex 0. Write the order of traversal 	58
6	Consider the given graph. Perform Depth First Search (DFS) traversal starting from vertex A. Show the stack contents at each step of the traversal and write the final DFS traversal order  	63



Module -III

SHORT ANSWERS

1. Define Linked List and how is it different from an array.

A linked list is a structure where each element (node) has data and a pointer to next node. It uses non-contiguous memory, unlike arrays which use continuous memory. Linked lists grow dynamically and allow easy insertions and deletions.

2. Define Singly Linked List.

A singly linked list is a data structure where each node contains data and a pointer to the next node. It allows sequential access from the first node to the last node. The last node points to NULL.

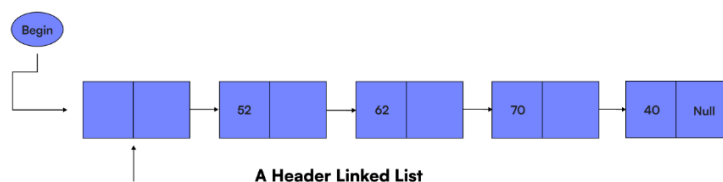
4. What is a Node in a Linked List?

A **Node** in a **Linked List** is the basic unit that stores **data** and a **reference (link)** to the next **node** in the list.

Each node contains two parts:

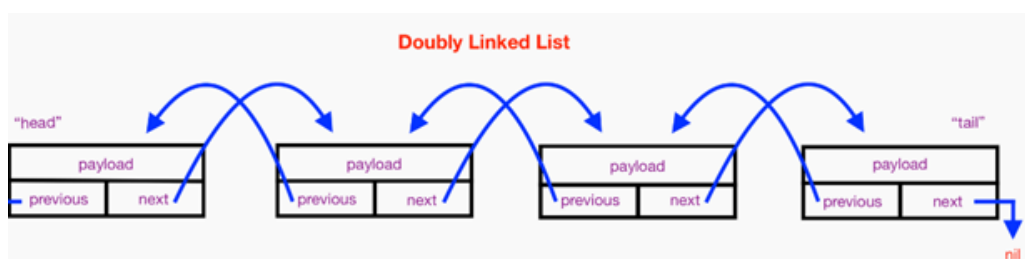
1. **Data field** – stores the actual value.
2. **Link (pointer)** – stores the address of the next node in the list.

The nodes are connected together through these links to form a **linked list**.



4. Define a Doubly Linked List.

A doubly linked list is a linear data structure where each node contains data, a pointer to the next node (next), and a pointer to the previous node (prev). This allows traversal in both forward and backward directions.





5. Define String in data structures

A **String** in Data Structures is a **finite sequence of characters arranged in a specific order**. These characters may **include** letters, digits, or special symbols, **and they are used to represent textual data**.

6. What is String Manipulation?

String manipulation refers to performing various operations on strings such as creating, modifying, searching, or transforming text in a program. These techniques help in processing textual data efficiently.

LONG ANSWERS

X. Discuss the process of traversing and searching elements in a linked list. Explain how nodes are accessed sequentially and provide a suitable example.

A **linked list** is a linear data structure in which elements (nodes) are connected using pointers.

Each node contains **two parts**:

1. **Data**
2. **Link (address of the next node)**

Unlike arrays, linked lists do **not support direct access** using index. Therefore, searching in a linked list is done using **linear search**.

Traversing a Linked List

Traversal in a linked list is the process of visiting each node one by one to access or display the data stored in the list. Since the elements in a linked list are not stored in contiguous memory locations, nodes are accessed sequentially using pointers (links) starting from the head node.

Traversing a Linked List Steps

1. Start from the **head node** of the linked list.
2. Access the **data** of the current node.
3. Move to the **next node using the link field**.
4. Repeat the process until the **next pointer becomes NULL**, which indicates the end of the list.



Searching Process in a Linked List:

The searching operation follows these steps:

Step 1

Start from the **first node (head)** of the linked list.

Step 2

Compare the **data of the current node** with the element to be searched (key).

Step 3

- If the data matches the key, the search is **successful** and the position is returned.
- If the data does not match, move to the **next node** using the link.

Step 4

Repeat the comparison for each node until:

- The element is found, or
- The end of the linked list is reached (NULL).

Step 5

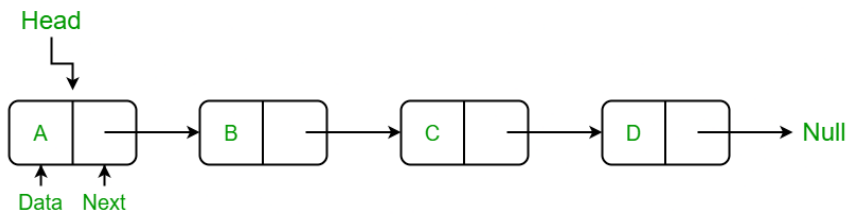
If the end is reached and the element is not found, the search is unsuccessful.

✗ Explain the concept of linked lists as a dynamic data structure. Describe how a linked list is represented in memory and illustrate its structure with a diagram.

A **Linked List** is a **dynamic data structure** in which elements (nodes) are stored in **non-contiguous memory locations** and connected using **links (pointers)**. Unlike arrays, the size of a linked list **can grow or shrink during program execution**, which makes it dynamic.

Each element in a linked list is called a **node**, and every node contains two parts:

1. Data part – stores the actual value
2. Link part – stores the address of the next node

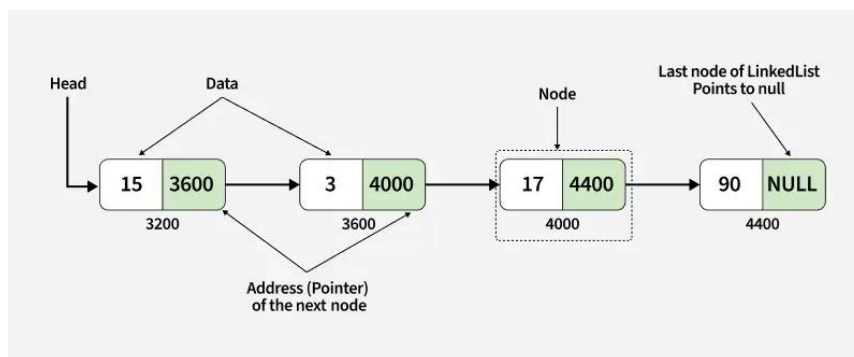


Representation in Memory:

Unlike arrays, linked list elements are not stored in consecutive memory locations. Nodes are connected using pointers, which allows dynamic memory allocation.

Features of Linked List

1. A linked list consists of a collection of nodes.
2. Each node contains data and a pointer to the next node.
3. Memory allocation is done dynamically at runtime.
4. Nodes are stored in non-contiguous memory locations.
5. The head node stores the address of the first node.
6. The last node contains NULL in its link part.
7. Elements are accessed sequentially, not randomly.



Advantages of Linked List

1. *Dynamic Size*
The size of a linked list can grow or shrink as needed.
2. *Efficient Insertion and Deletion*
Elements can be inserted or deleted without shifting other elements.
3. *Better Memory Utilization*
Memory is allocated only when required, reducing wastage.
4. *Flexibility*
Linked lists are useful in implementing stacks, queues, graphs, and trees.
5. *No Fixed Size Limitation*
Unlike arrays, linked lists do not require a predefined size.



✗ Describe the insertion operation in a linked list at different positions such as beginning, middle and end. Explain the procedure with examples.

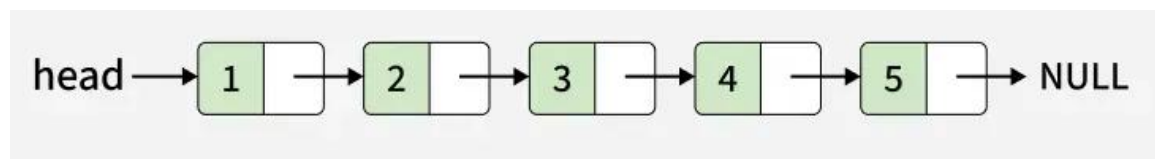
Structure of a Node:

In a singly linked list, each element is represented by a node. A node is a self-referential structure consisting of two fields:

1. Data Field
 - Stores the actual information or value of the node.
 - It may be an integer, character, or any data type.
2. Link Field (Next Pointer)
 - Stores the address of the next node in the linked list.
 - It connects the current node to the next node.

The last node of the linked list has its Next field set to NULL, which indicates the end of the list.

The address of the first node is stored in a pointer called Head.



Singly Linked List Representation

- Nodes are stored in non-contiguous memory locations
- Nodes are connected using pointers

Insertion into a Singly Linked List

Insertion is the process of adding a new node into the linked list.

In a singly linked list, insertion can be performed at three different positions:

1. Insertion at the beginning
2. Insertion at the end
3. Insertion at a specific position (middle)

1. Insertion at the Beginning

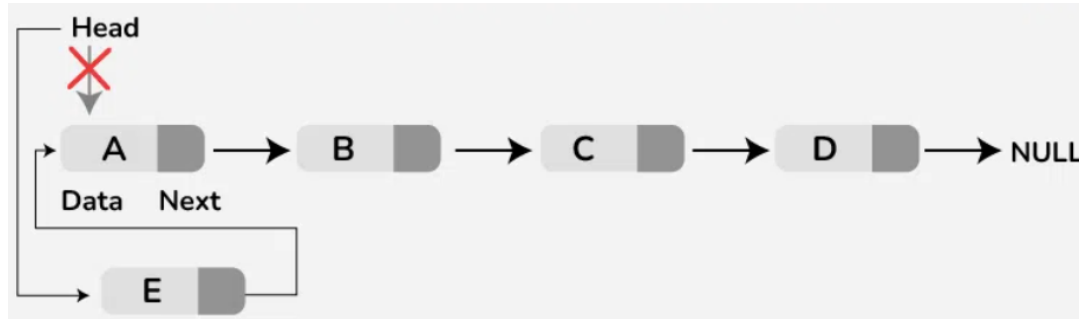
In this method, a new node is inserted before the first node.

Steps:

1. Create a new node.
2. Store the data in the new node.



3. Make the new node's Next point to the current head.
4. Update the head to point to the new node.



Result:

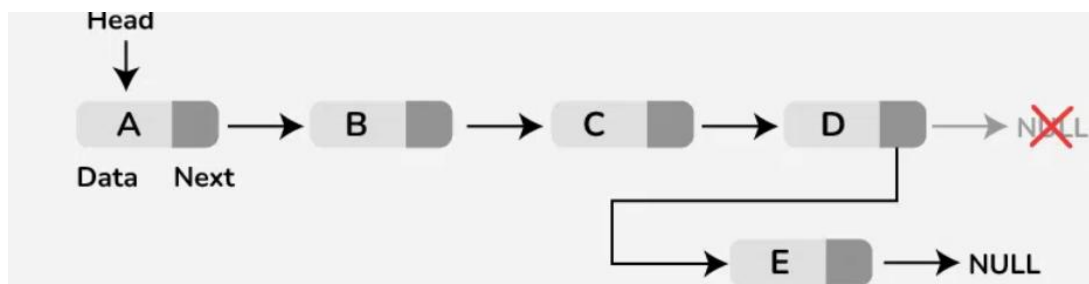
The new node becomes the first node of the list.

2. Insertion at the End

In this method, a new node is inserted after the last node.

Steps:

1. Create a new node.
2. Store the data in the new node and set Next = NULL.
3. Traverse the linked list until the last node is reached.
4. Make the last node's Next point to the new node.



Result:

The new node becomes the last node of the list.

3. Insertion at a Specific Position (Middle)

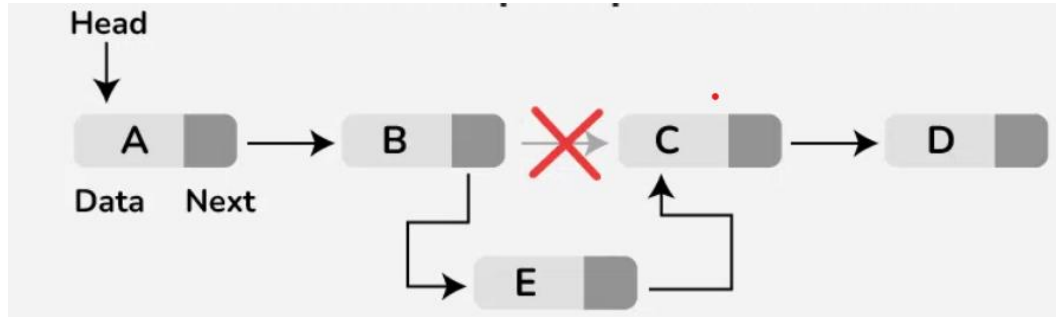
In this method, a new node is inserted at a given position in the list.

Steps:

1. Create a new node and store the data.
2. Traverse the list to the node after which insertion is required.



3. Set the new node's Next to the next node.
4. Update the previous node's Next to the new node.



Result:

The new node is inserted at the required position.

Advantages of Insertion in Linked List

- No shifting of elements is required
- Memory allocation is dynamic
- Efficient compared to array insertion

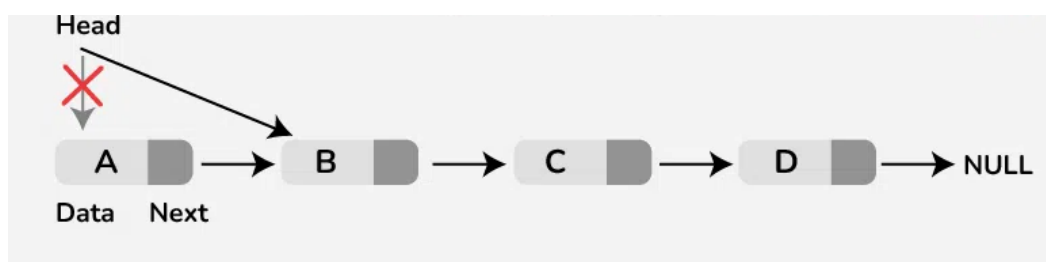
✓4. Explain the deletion operation in a linked list. Discuss the steps involved in deleting nodes from different positions in the list.

A **linked list** is a linear data structure where each element (node) contains **data** and a **pointer to the next node**. Deletion means removing a node from the list while maintaining the links so that the list remains connected.

Types of deletion

1. Deletion at the beginning

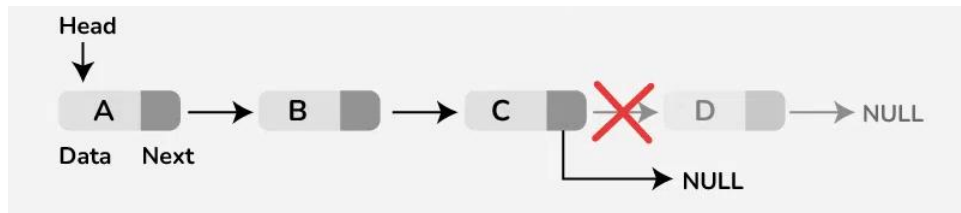
- If the node to be deleted is the first node, update the **head** pointer to point to the second node, then free the memory of the old first node.





2. Deletion from the end

- Traverse to the **second-last node**, set its next to NULL, then free the last node (no longer reachable).



2. Deletion from middle / at a position

- Traverse to the node just **before** the target node, change its next to skip the target, then free the target node.



Example

Consider a singly linked list:

10 → 20 → 30 → 40 → NULL

- **Delete node 20 (middle)**
 - Start from head (10).
 - Move to node 10; its next points to 20 (to be deleted).
 - Set 10→next = 30.
 - Now the list becomes:

10 → 30 → 40 → NULL

Node 20 is removed from the chain.



✓5. Discuss the concept of strings in data structures. Explain various string manipulation techniques used in programming.

A string is a sequence of characters used to represent text data such as words, sentences, or symbols. Characters in a string may include letters, numbers, spaces, and special symbols. Strings are widely used in programming for storing and processing textual information.

Example:
"Data Structures"

String manipulation refers to performing various operations on strings such as creating, modifying, searching, or transforming text in a program. These techniques help in processing textual data efficiently.

Common String Manipulation Techniques

1. Concatenation

Concatenation means joining two or more strings to form a single string.

Example:
"Data" + "Structures" = "DataStructures"

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s1 = "Data";
    string s2 = "Structures";
    cout << s1 + " " + s2 << endl;
}
```

Output
Data Structures

2. Substring Extraction

A substring is a part of a string obtained from the original string.

Example

String = "Programming"

Substring = "Program"

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s = "Programming";
    cout << s.substr(0,7) << endl;
}
```



3. String Comparison

String comparison checks whether two strings are equal.

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s1 = "Hello";
    string s2 = "Hello";
    if(s1 == s2){
        cout << "Strings are equal" << endl;
    }
}
```

4. Searching in a String

Searching means finding whether a character or word exists inside a string.

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s = "Data Structures";
    if(s.find("Data") != string::npos)
        cout << "True" << endl;
    else
        cout << "False" << endl;
}
```

True

5. Replace Operation

Replacing means substituting one substring with another substring.

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s = "Data Structure";
    s.replace(s.find("Structure"), 9, "Structures");
    cout << s << endl;
}
```

Data Structures

Applications of String Manipulation

- Text processing
- Search engines
- Data analysis
- Natural language processing
- Compilers and editors

6. Explain the structure and advantages of a Doubly Linked List.



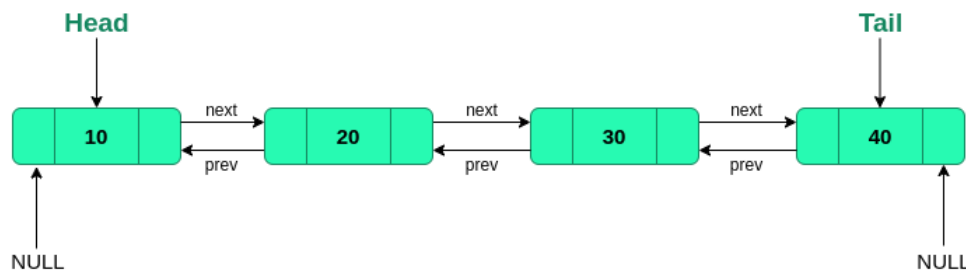
A **doubly linked list (DLL)** is a linked list where each node contains **three fields**:

- data: stores the value.
- prev: points to the **previous node**.
- next: points to the **next node**.

The list has two pointers:

- head points to the first node.
- tail (optional) points to the last node.

Doubly Linked List



Advantages

- Bidirectional traversal: You can traverse in both forward and backward directions using next and prev pointers.
- Efficient insertion/deletion at both ends: Insertion and deletion at the beginning and end can be done in $O(1)$ time.
- Easy node deletion: If a node is given, its prev and next pointers allow deletion in $O(1)$ by updating neighboring links.
- Supports reverse operations: Reversing the list or implementing applications like undo–redo, browser history, and music playlists is easier.

Main disadvantage is **higher memory usage** per node (two pointers instead of one).

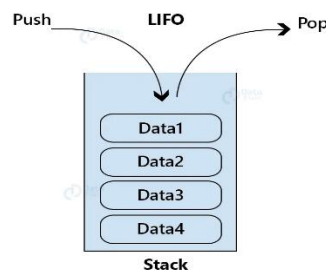


Module -IV

SHORT ANSWERS

✓1. What is a Stack?

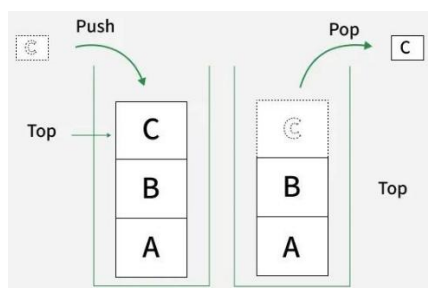
A stack is a linear data structure that follows the LIFO (Last In First Out) or FILO (First In Last Out) principle. Insertion (push) and deletion (pop) are allowed only at one end, called the top of the stack.



✓2. What are the operations performed on Stack.?

The main operations performed on a Stack are:

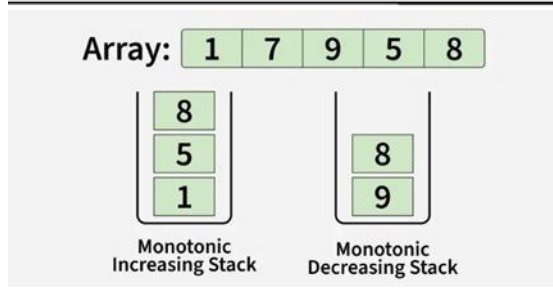
1. **Push** – Inserting an element onto the top of the stack.
2. **Pop** – Removing the top element from the stack.
3. **Peek (or Top)** – Viewing the top element without removing it.
4. **isEmpty** – Checking whether the stack is empty.
5. **isFull** – Checking whether the stack is full (in array implementation).



✓3. What is a Monotonic Stack ? What is Next Greater Element problem?

Monotonic Stack:

A Monotonic Stack is a stack in which elements are maintained in increasing or decreasing order. It is used to efficiently solve problems involving next greater or smaller elements.



Next Greater Element Problem:

The Next Greater Element (NGE) problem finds the first greater element to the right of each element in an array. If no greater element exists, the result is -1.

✓4. Define Infix, Prefix and Postfix expressions.

Infix Expression:

An expression in which the **operator is written between the operands.**

Example: $A + B$

Prefix Expression:

An expression in which the **operator is written before the operands.**

Example: $+ A B$

Postfix Expression:

An expression in which the **operator is written after the operands.**

Example: $A B +$

✓4. Define Queue and its operations

A **Queue** is a linear data structure that follows the **FIFO (First In First Out)** principle, where the element inserted first is removed first.



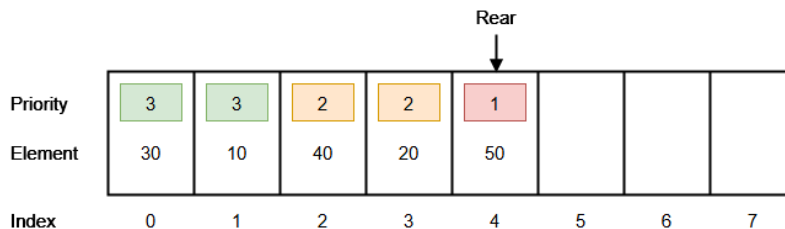
Operations of Queue:

1. **Enqueue** – Inserting an element at the rear of the queue.
2. **Dequeue** – Removing an element from the front of the queue.
3. **Peek (Front)** – Viewing the front element without removing it.
4. **isEmpty** – Checking whether the queue is empty.
5. **isFull** – Checking whether the queue is full.



✓ What is a Priority Queue?

A priority queue is an abstract data structure where each element is associated with a priority. Elements with higher priority are removed first, regardless of the order of insertion; it can be implemented using a heap or other ordered structures.

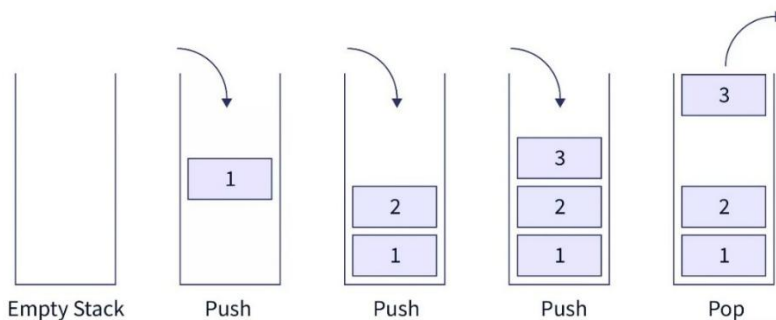


LONG ANSWERS

1. Explain the concept of stack data structure and Describe its basic operations such as push and pop with suitable example

A stack is a linear data structure that follows the LIFO (Last In, First Out) or FILO (First In, Last Out) principle. In stack operations, elements can be inserted and deleted only from one end, known as the top of the stack. Insertion at the top is called push, and deletion from the top is called pop.

Think of a stack of plates: when you add a plate, you place it on top; when you remove a plate, you remove the topmost one. This is exactly how a stack behaves in programming.





Structure of a stack

- A stack can be implemented using an array or a linked list.
- A pointer (or index) TOP is maintained to keep track of the top element.
- Initially TOP = -1 indicates an empty stack. On each push, TOP is incremented; on each pop, TOP is decremented.

Basic operations of a stack:

1. **Push(x)** – Insert an element
 - Adds an element x to the **top** of the stack.
 - Before pushing, check if the stack is full (in array-based implementation) to avoid **stack overflow**.
 - Change: increment TOP and store x at position TOP.
 2. **Pop()** – Delete an element
 - Removes and returns the element from the **top** of the stack.
 - Before popping, check if the stack is empty to avoid **stack underflow**.
 - Change: read the element at TOP and decrement TOP.
 3. **Peek / Top()** – View the top element
 - Returns the value of the top element **without removing it**.
 - Useful for checking the current state of the stack.
 4. **isEmpty()** – Check if stack is empty
 - Returns true if TOP = -1 (array-based) or the top pointer is NULL (linked-list-based).
 - Used to control pop and peek operations.
 5. **isFull()** – Check if stack is full (array-based)
 - Returns true if TOP = size-1, indicating no more elements can be inserted.
 - Applies only when the stack is implemented using a fixed size array.
 6. **Size()** – Return number of elements
 - Returns the current count of elements in the stack (often TOP + 1 in array-based stacks).
- ❖ All core operations (**push**, **pop**, **peek**, **isEmpty**) take **O(1) time** in both array and linked-list implementations, making stack very efficient for certain applications.



2. Write a program to implement stack operations.

```
#include <bits/stdc++.h>
using namespace std;
vector<int> s;
void push(int x){
    s.push_back(x);
    cout<<"Pushed: "<<x<<endl;
}
void pop(){
    if(s.empty()){
        cout<<"Stack Underflow"<<endl;
        return;
    }
    cout<<"Popped: "<<s.back()<<endl;
    s.pop_back();
}
void peek(){
    if(s.empty()){
        cout<<"Stack is Empty"<<endl;
        return;
    }
    cout<<"Top element: "<<s.back()<<endl;
}
void display(){
    if(s.empty()){
        cout<<"Stack is Empty"<<endl;
        return;
    }
    cout<<"Stack: ";
    for(int i=s.size()-1;i>=0;i--)
        cout<<s[i]<<" ";
    cout<<endl;
}
int main(){
    push(10);
    push(20);
    push(30);
    display();
    pop();
    peek();
    display();
}
```

Output

```
Pushed: 10
Pushed: 20
Pushed: 30
Stack: 30 20 10
Popped: 30
Top element: 20
Stack: 20 10
```



3. Convert the following expressions into in to postfix form using stack.

a) $A - B / C * D + E$

b) $(A + B) * C + (D - E) / F$

To convert an infix expression to postfix expression, we use a stack and follow operator precedence rules.

Operator precedence: * has higher precedence than +.

Algorithm Steps

1. Scan the infix expression from left to right.
2. If the symbol is an **operand**, add it to the postfix expression.
3. If the symbol is an **operator**, push it into the stack according to precedence.
4. If the stack contains an operator with higher precedence, pop it to postfix.
5. After scanning the expression, pop all remaining operators from the stack.

Conversion Steps

Convert the expression $A + B * C$ into postfix form using stack

Infix Expression:

a) $A - B / C * D + E$

Stepwise Conversion

Scan	Stack	Postfix
A		A
-	-	A
B	-	AB
/	- /	AB
C	- /	ABC
*	- *	ABC/
D	- *	ABC/D
+	+	ABC/D*-
E	+	ABC/D*-E
End		ABC/D-E+*

Postfix Expression

ABC/D*-E+



b) $(A+B) * C + (D-E) / F$

Stepwise Conversion

Scan	Stack	Postfix
(	(	
A	(	A
+	(+	A
B	(+	AB
)		AB+
*	*	AB+
C	*	AB+C
+	+	AB+C*
(	+ (	AB+C*
D	+ (	AB+C*D
-	+ (-	AB+C*D
E	+ (-	AB+C*DE
)	+	AB+C*DE-
/	+ /	AB+C*DE-
F	+ /	AB+C*DE-F
End		AB+C*DE-F/+

Postfix Expression :

AB+C*DE-F/+

4. Explain the applications of stack in computer science.

Applications of Stack in Computer Science:

1. Function call management (Call stack)

- The call stack stores activation records for active functions.
- Each record holds return address, local variables, parameters, and control state.



- When a function finishes, its record is popped and control returns to the caller.
- This ensures correct nested execution and supports automatic memory allocation for local variables.

2. Recursion

- Each recursive call is pushed onto the stack as a new activation frame.
- The recursion proceeds until the base case is reached.
- Then frames are popped in reverse order, calculating and returning values step by step.
- Examples: factorial, Fibonacci, tree traversals, etc.

3. Expression evaluation and conversion

- Infix to postfix / prefix:
 - Operators are pushed/popped based on precedence and associativity.
 - Parentheses are handled using stack to ensure correct grouping.
- Postfix evaluation:
 - Operands are pushed; when an operator is found, two operands are popped, the operation is done, and the result is pushed back.
- Used in compilers, calculators, and query evaluators.

4. Syntax checking (parentheses, brackets, braces)

- Opening symbols (, [, { are pushed onto the stack.
- Closing symbols),], } cause the stack to be popped, and the popped symbol must match.
- If the stack becomes empty and all matches are correct, the expression is balanced and syntactically valid.
- Used in compilers, IDEs, and HTML/XML parsers.

5. Undo-redo operations

- Each user action (typing, deletion, formatting, etc.) is pushed onto an undo stack.
- When the user presses Undo, the top action is popped and reversed.
- A redo stack often stores undone actions so they can be redone if needed.
- Common in text editors, graphic editors, and IDEs.

6. Backtracking algorithms

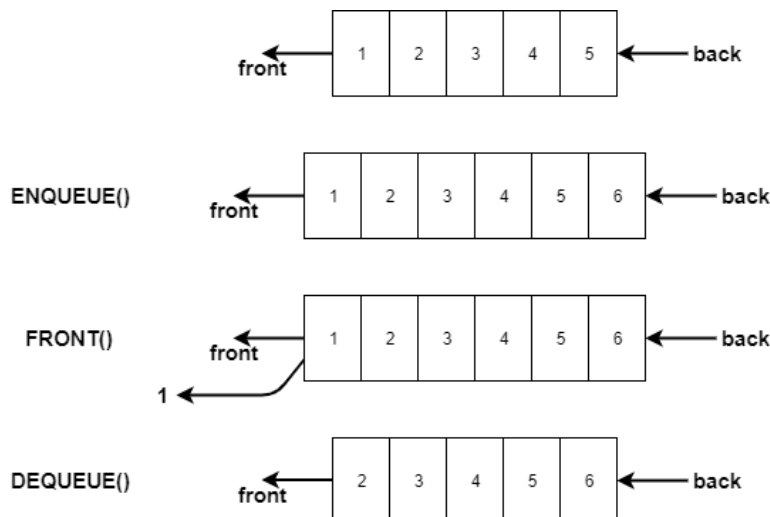


- Depth-First Search (DFS) in graphs and trees uses a stack to store the current path.
- When a path dies (dead end), the algorithm pops back to the last valid state and tries another branch.
- Applied in maze solving, N-queens problem, subset generation, and other backtracking-based problems.

5. Explain the concept of Queue data structure and describe its operations such as enqueue and dequeue with suitable example.

A queue is a linear data structure that follows the FIFO (First In First Out) principle: the first element inserted is the first one to be removed. Operations take place at two ends:

- Front: elements are removed from here.
- Rear (back): elements are inserted here.



Basic queue operations

1. Enqueue (insert)

- Adds an element to the rear of the queue.
- Before enqueueing, check if the queue is full (for array-based queues) to avoid queue overflow.

2. Dequeue (delete)

- Removes and returns the element from the front of the queue.
- Before dequeuing, check if the queue is empty; otherwise queue underflow occurs.

3. Peek / Front (view front element)



- Returns the element at the front without removing it.
- Used to inspect the next element to be processed.

4. isEmpty

- Returns true if no elements are present.
- Used to prevent invalid dequeue operations.

5. isFull (array-based)

- Returns true if the queue has reached its maximum capacity.
- Used to prevent overflow during enqueue.

Example with values

Consider a queue that stores integers and initially is empty:

Operation	Queue State / Result
enqueue(10)	queue: [10]
enqueue(20)	queue: [10 20]
enqueue(30)	queue: [10 20 30]
dequeue()	removes 10 → queue: [20 30]
dequeue()	removes 20 → queue: [30]
peek()	returns 30 (front element)
isEmpty()	false
dequeue()	removes 30 → queue: []
isEmpty()	true

This behaviour is similar to a ticket counter or ATM line where the first person to join the line is the first to be served.

Real-life analogy

- Bus stop queue: People join at the rear and leave from the front.
- CPU scheduling: Ready processes are stored in a queue; the first one to arrive gets CPU time first.



6. Write a program to implement QUEUE operations.

```
#include <bits/stdc++.h>
using namespace std;
vector<int> q;
void enqueue(int x){
    q.push_back(x);
    cout<<"Enqueued: "<<x<<endl;
}
void dequeue(){
    if(q.empty()){
        cout<<"Queue Underflow"<<endl;
        return;
    }
    cout<<"Dequeued: "<<q.front()<<endl;
    q.erase(q.begin());
}
void front_element(){
    if(q.empty()){
        cout<<"Queue is Empty"<<endl;
        return;
    }
    cout<<"Front element: "<<q.front()<<endl;
}
void display(){
    if(q.empty()){
        cout<<"Queue is Empty"<<endl;
        return;
    }
    cout<<"Queue: ";
    for(int x : q)
        cout<<x<<" ";
    cout<<endl;
}
int main(){
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    front_element();
    display();
}
```

Example Output

```
Enqueued: 10
Enqueued: 20
Enqueued: 30
Queue: 10 20 30
Dequeued: 10
Front element: 20
Queue: 20 30
```

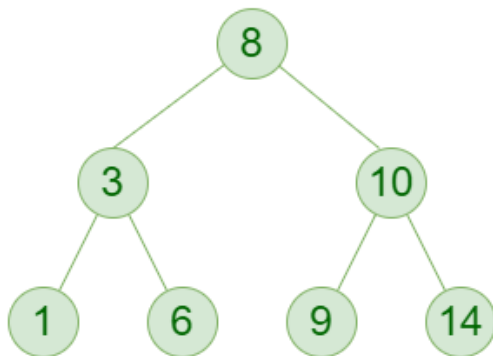


MODULE -V

SHORT ANSWERS

1. Define Binary Tree.

A binary tree is a hierarchical data structure in which each node has at most two children, called the left child and the right child. The topmost node is called the root, and nodes with no children are called leaf nodes.

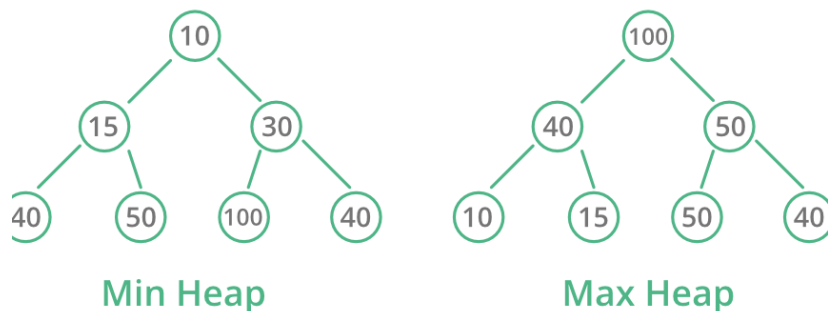


2. What is a Binary Search Tree?

A Binary Search Tree (BST) is a binary tree in which for every node, all elements in the left subtree are less than the node's value and all elements in the right subtree are greater than the node's value. This ordering property enables efficient search, insertion, and deletion in $O(\log n)$ average time.

3. What is a Heap in Data Structures?

A heap is a complete binary tree that satisfies the heap property. In a max-heap, the parent node is greater than or equal to its children; in a min-heap, it is less than or equal to them. Heaps are used to implement priority queues and efficient sorting algorithms like heapsort.





4. What is Breadth First Search?

Breadth First Search (BFS) is a graph traversal algorithm that visits all nodes at the current depth level before moving to the next level. It uses a queue to keep track of nodes to be visited, starting from a given **source node**.

5. What is Depth First Search?

Depth First Search (DFS) is a graph traversal algorithm that explores as far as possible along one branch before backtracking. It uses a stack (or recursion) to store the path and visits nodes in preorder, inorder, or postorder depending on the implementation.

6. Define Graph.

A graph is a non-linear data structure consisting of a finite set of vertices (nodes) and a set of edges connecting them. Graphs can be directed or undirected and are used to model relationships such as networks, maps, and dependencies

LONG ANSWERS

1. Construct a Binary Search Tree for the elements 45, 15, 79, 90, 10, 55, 12, 20, 50.

Binary Search Tree Creation

A Binary Search Tree is a binary tree in which the left child contains values smaller than the parent node and the right child contains values greater than the parent node.

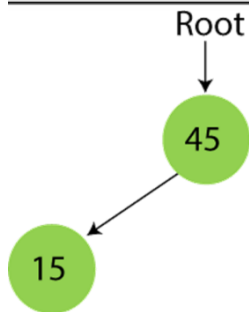
From given data Consider the elements:
45, 15, 79, 90, 10, 55, 12, 20, 50

Step 1: Insert **45** as the **root node** of the tree.



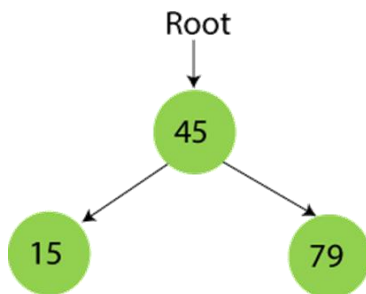
Step 2: Insert 15.

Since $15 < 45$, it is inserted as the **left child of 45**.



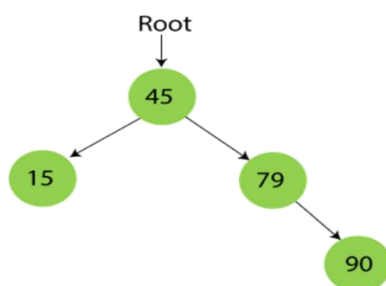
Step 3: Insert 79.

Since $79 > 45$, it is inserted as the right child of 45.



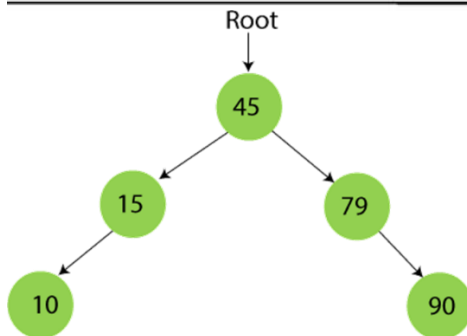
Step 4: Insert 90.

Since $90 > 45$ and $90 > 79$, it is inserted as the **right child of 79**.



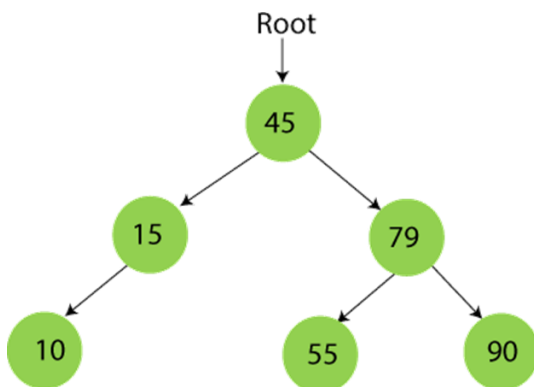
Step 5: Insert 10.

Since $10 < 45$ and $10 < 15$, it is inserted as the **left child of 15**.



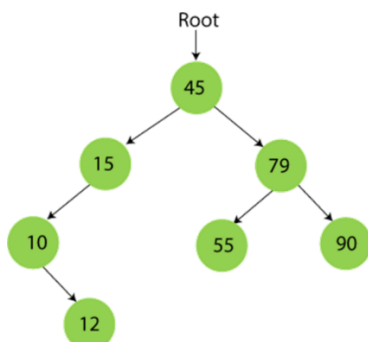
Step 6: Insert 55.

Since $55 > 45$ and $55 < 79$, it is inserted as the **left child of 79**.



Step 7: Insert 12.

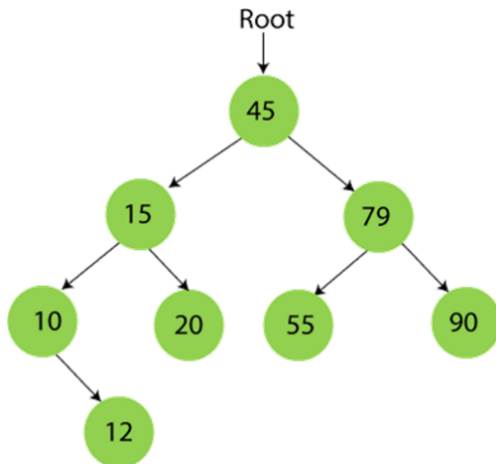
Since $12 < 45$, $12 < 15$, but $12 > 10$, it is inserted as the **right child of 10**.





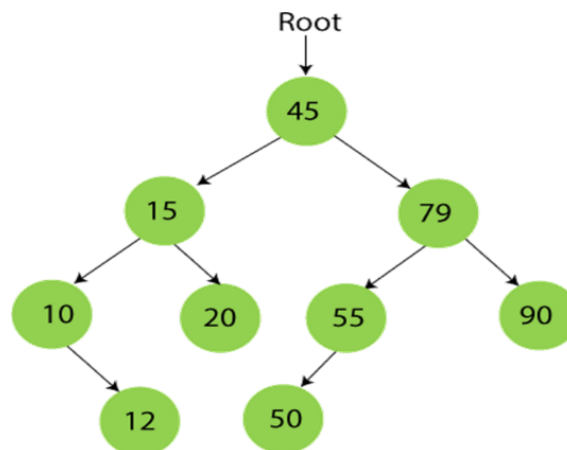
Step 8: Insert 20.

Since $20 < 45$ and $20 > 15$, it is inserted as the **right child of 15**.



Step 9: Insert 50.

Since $50 > 45$, $50 < 79$, and $50 < 55$, it is inserted as the **left child of 55**





2. Write a Python program to create a Max Heap from a given list of elements and display the heap.

A Max Heap is a complete binary tree where the parent node is always greater than or equal to its child nodes.

```
he.py > ...
1  # Function to heapify a subtree rooted at index i
2  def heapify(arr, n, i):
3      largest = i
4      left = 2*i + 1
5      right = 2*i + 2
6
7      if left < n and arr[left] > arr[largest]:
8          largest = left
9
10     if right < n and arr[right] > arr[largest]:
11         largest = right
12
13     if largest != i:
14         arr[i], arr[largest] = arr[largest], arr[i]
15         heapify(arr, n, largest)
16
17     # Function to build Max Heap
18     def build_max_heap(arr):
19         n = len(arr)
20         for i in range(n//2 - 1, -1, -1):
21             heapify(arr, n, i)
22
23     # User input
24     elements = list(map(int, input("Enter elements separated by space: ").split()))
25
26     build_max_heap(elements)
27
28     print("Max Heap:", elements)
```

Example Run

```
|
| Enter elements separated by space: |
| 45 36 54 27 63 72 61 18          |
|
| Max Heap: [72, 63, 61, 27, 36, 54, |
|              45, 18]              |
|
```



3.Explain the concept of Heap in data structures.

A heap is a tree-based data structure that is typically implemented as a complete binary tree and satisfies a special heap property. It is widely used to implement priority queues and to design efficient algorithms such as heapsort. A heap is commonly used to implement **priority queues** and is also used in algorithms such as **Heap Sort**.

Properties of a Heap

1. *Complete Binary Tree*

A heap must be a complete binary tree. This means all levels of the tree are completely filled except possibly the last level, which is filled from left to right.

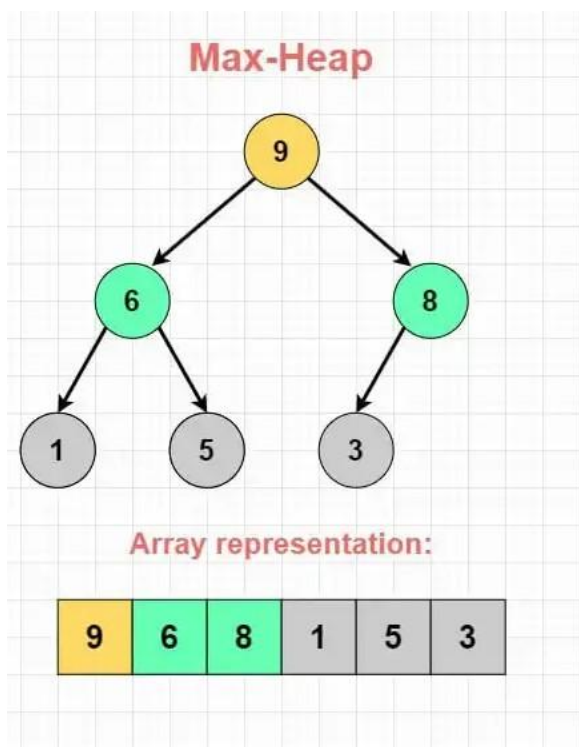
2. *Heap Property*

The value of the parent node is related to its children according to the heap type.

Types of Heap

1. Max Heap:

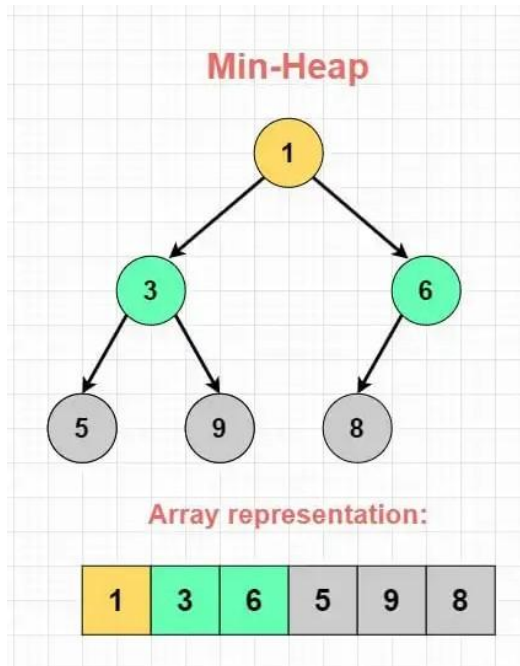
In a Max Heap, the value of the parent node is greater than or equal to its child nodes. The largest element is always at the root.





2. Min Heap:

In a Min Heap, the value of the parent node is smaller than or equal to its child nodes. The smallest element is always at the root.



Array Representation of Heap

A heap can be stored efficiently in an **array**.

If a node is at index **i**:

Left child = $2i + 1$

Right child = $2i + 2$

Parent = $(i - 1) / 2$

Applications of Heap

- Priority queues
- Heap sort algorithm
- Graph algorithms such as Dijkstra's algorithm
- CPU scheduling
- Network routing



4. Build a Max Heap from the elements 45, 36, 54, 27, 63, 72, 61, 18.

Ans:

Creation of Max Heap

A Max Heap is a complete binary tree in which the value of the parent node is greater than or equal to the values of its children.

From given data Consider the elements:

45, 36, 54, 27, 63, 72, 61, 18

These elements are inserted one by one and heap property is maintained after each insertion.

Step 1

Insert **45** as the root node.

Step 2

Insert **36**.

Since $36 < 45$, it becomes the **left child of 45**.

Step 3

Insert **54**.

Since $54 > 45$, swap them to maintain the **Max Heap property**.

Now **54 becomes the root**.

Step 4

Insert **27** as the next available child position.

Step 5

Insert **63**.

Since $63 > 36$ and $63 > 54$, it moves upward to maintain heap property.

Step 6

Insert **72**.

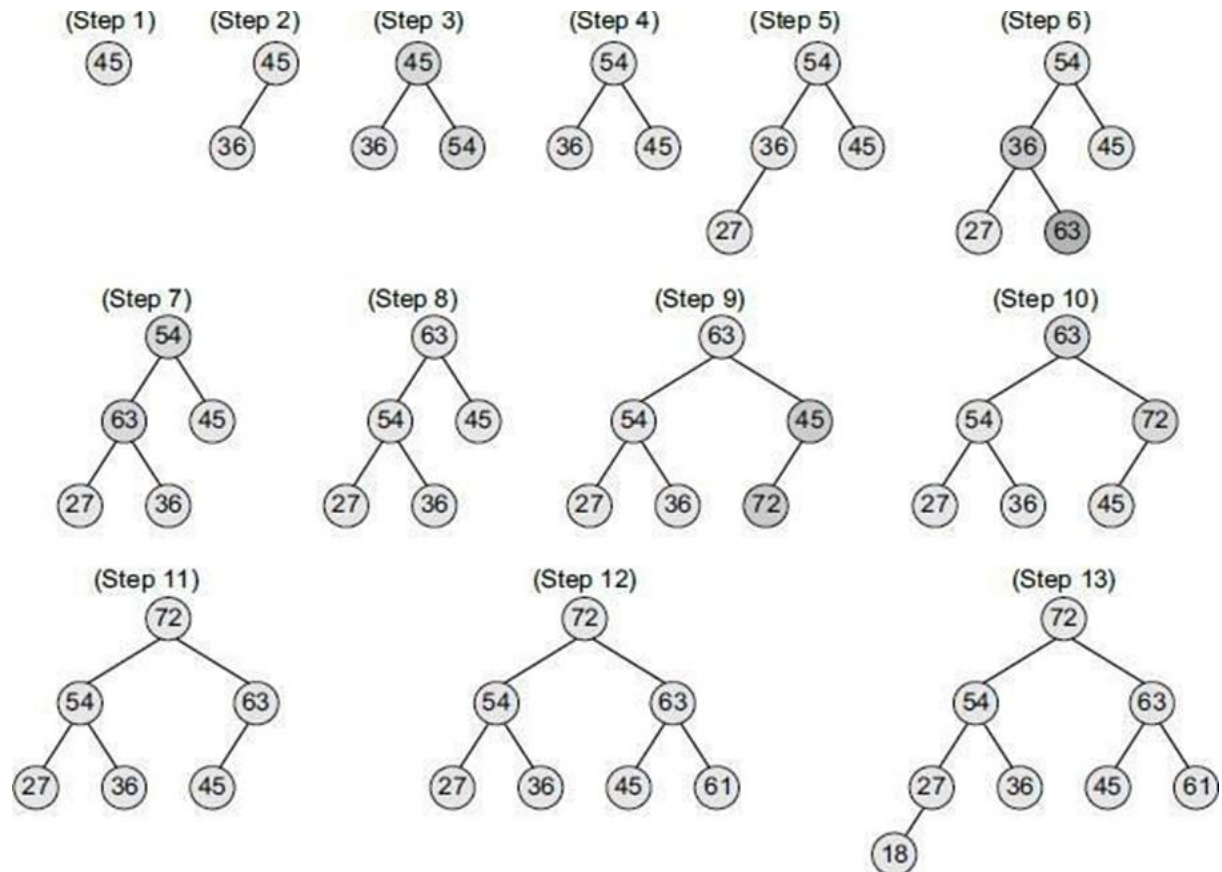
Since **72 is greater than its parent**, swap until it becomes the **root**.

Step 7: Insert **61** and adjust the heap if required.



Step 8

Insert **18** as the next leaf node.



Final Max Heap

Root: **72**

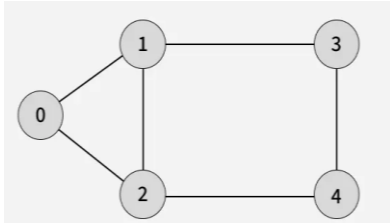
Left subtree: **54, 27, 36**

Right subtree: **63, 45, 61, 18**

This satisfies the Max Heap property where every parent node is greater than its children.



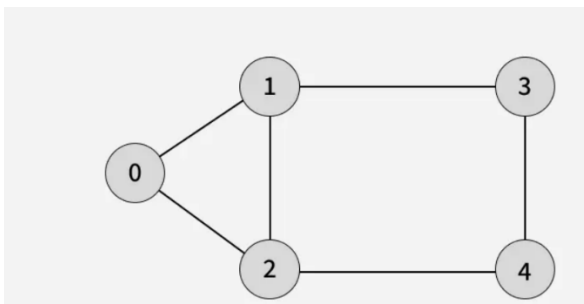
5. Consider the graph given below. Perform Breadth First Search (BFS) traversal starting from vertex 0. Write the order of traversal



Breadth First Search:

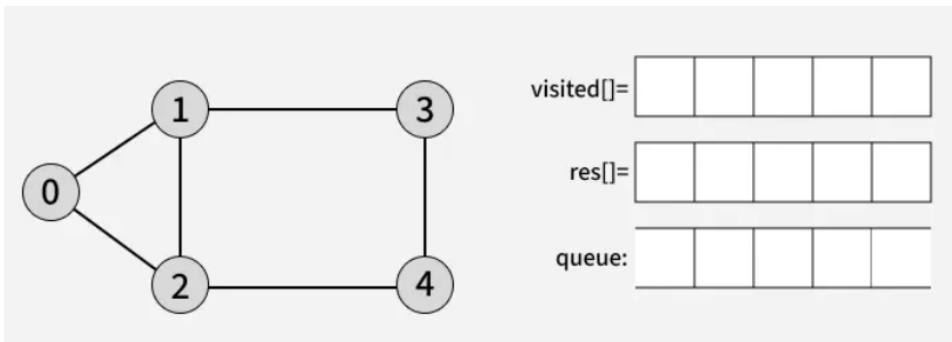
Breadth First Search is a graph traversal technique that visits vertices level by level. It uses a queue data structure to keep track of vertices to be processed.

From the given graph, start traversal from vertex 0.



Step 1

Create a **visited** array and an empty **queue**.





Step 2

Start with vertex **0**.

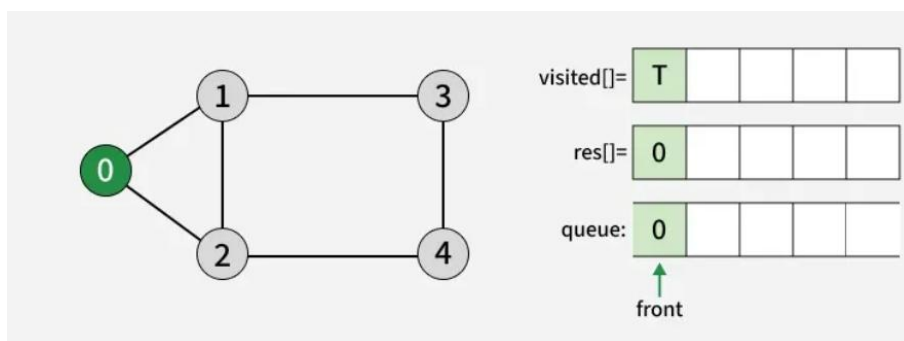
Mark it as visited and insert it into the queue.

Queue:

0

Result:

0



Step 3

Remove **0** from the queue and visit its adjacent vertices **1** and **2**.

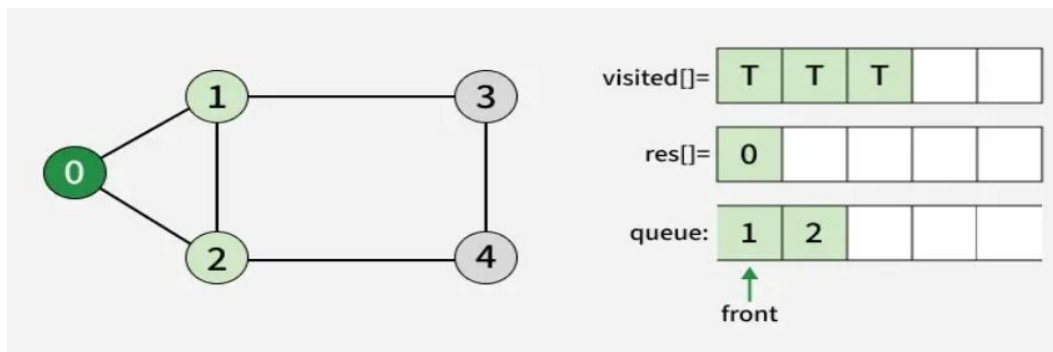
Mark them as visited and insert them into the queue.

Queue:

1, 2

Result:

0



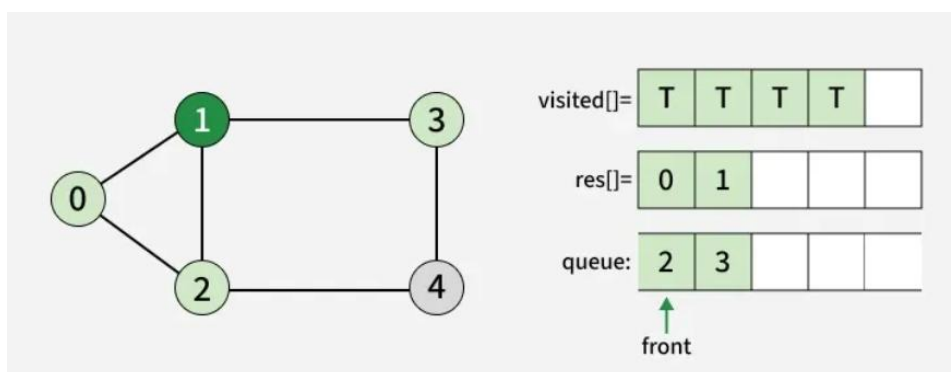


Step 4

Remove **1** from the queue and visit its unvisited adjacent vertex **3**.

Queue:
2, 3

Result:
0, 1

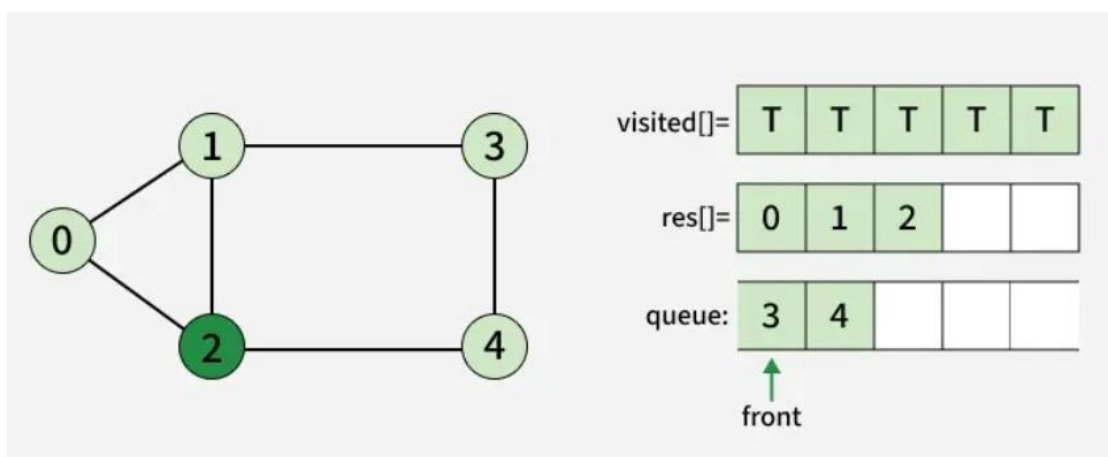


Step 5

Remove **2** from the queue and visit its unvisited adjacent vertex **4**.

Queue:
3, 4

Result:
0, 1, 2





Step 6

Remove 3 from the queue.

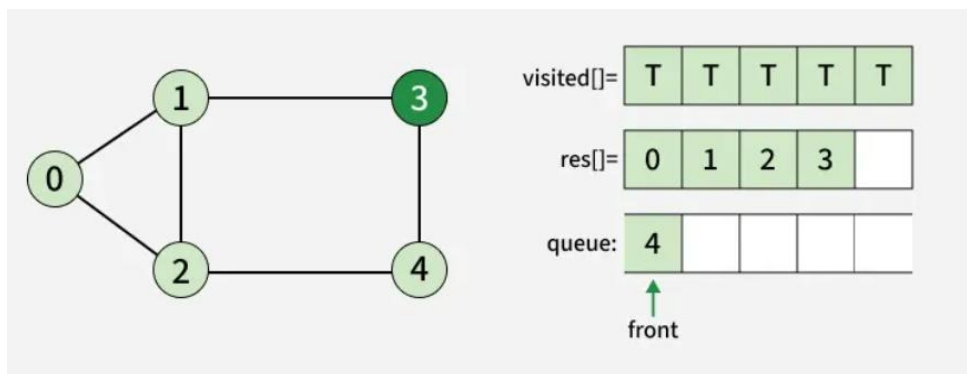
All adjacent vertices are already visited.

Queue:

4

Result:

0, 1, 2, 3



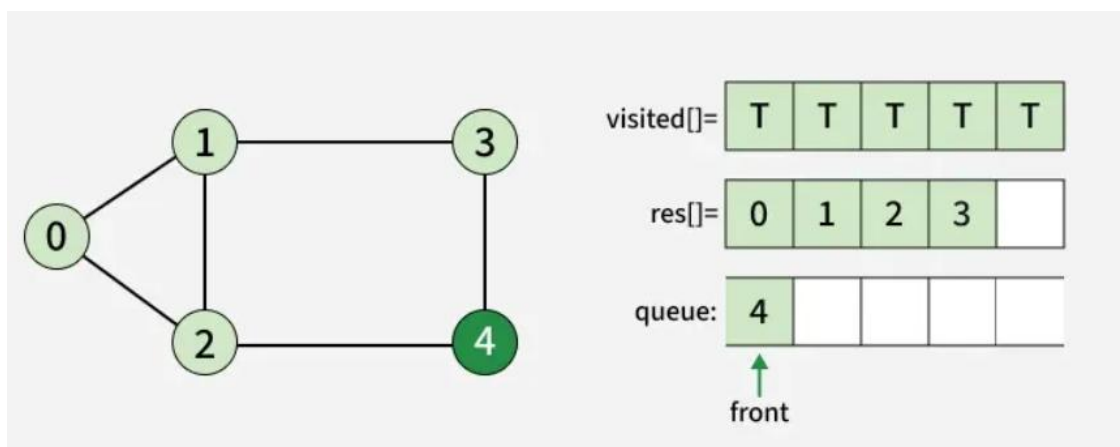
Step 7

Remove 4 from the queue.

All vertices are visited and the queue becomes empty.

Result:

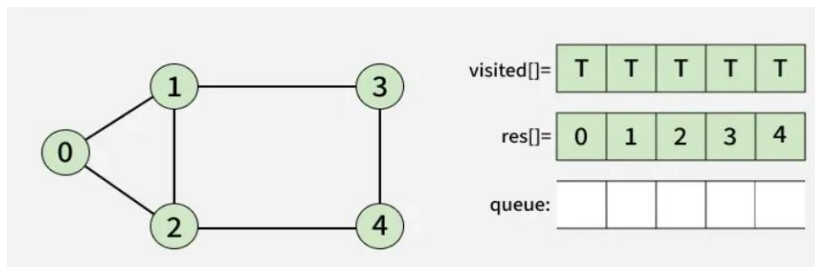
0, 1, 2, 3, 4



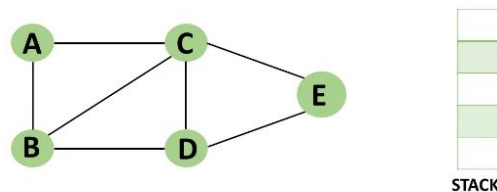


Final BFS Traversal Order

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$



6. Consider the given graph. Perform Depth First Search (DFS) traversal starting from vertex A. Show the stack contents at each step of the traversal and write the final DFS traversal order.



Depth-First Search Algorithm:

The depth-first search or DFS algorithm traverses or explores data structures, such as trees and graphs. The algorithm starts at the root node (in the case of a graph, you can use any random node as the root node) and examines each branch as far as possible before backtracking.

To implement DFS traversal, you need to take the following stages.

Step 1: Create a stack with the total number of vertices in the graph as the size.

Step 2: Choose any vertex as the traversal's beginning point. Push a visit to that vertex and add it to the stack.

Step 3 - Push any non-visited adjacent vertices of a vertex at the top of the stack to the top of the stack.



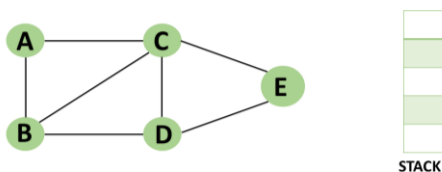
Step 4 - Repeat steps 3 and 4 until there are no more vertices to visit from the vertex at the top of the stack.

Step 5 - If there are no new vertices to visit, go back and pop one from the stack using backtracking.

Step 6 - Continue using steps 3, 4, and 5 until the stack is empty.

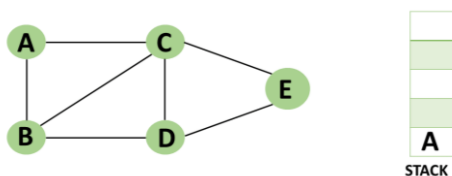
Step 7 - When the stack is entirely unoccupied, create the final spanning tree by deleting the graph's unused edges.

Consider the given graph : Starting vertex: A



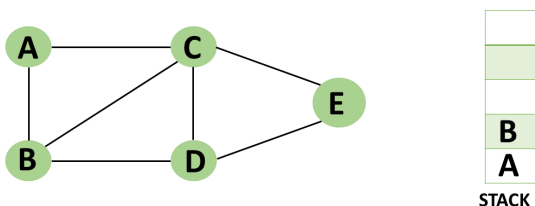
Step 1: Mark vertex A as a visited source node by selecting it as a source node.

- You should push vertex A to the top of the stack.



Step 2: Any nearby unvisited vertex of vertex A, say B, should be visited.

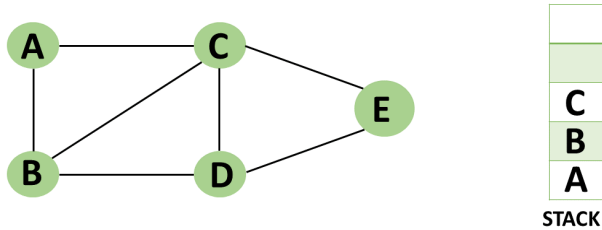
- You should push vertex B to the top of the stack.



Step 3: From vertex C and D, visit any adjacent unvisited vertices of vertex B. Imagine you have chosen vertex C, and you want to make C a visited vertex.

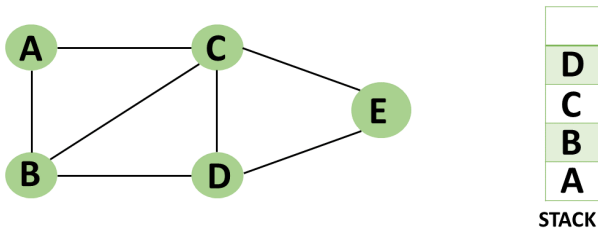


- Vertex C is pushed to the top of the stack.



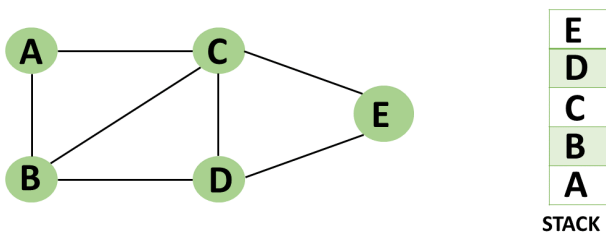
Step 4: You can visit any nearby unvisited vertices of vertex C, you need to select vertex D and designate it as a visited vertex.

- Vertex D is pushed to the top of the stack.

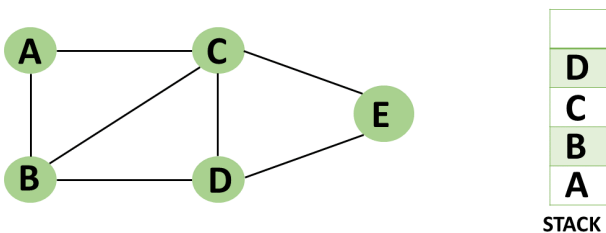


Step 5: Vertex E is the lone unvisited adjacent vertex of vertex D, thus marking it as visited.

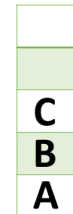
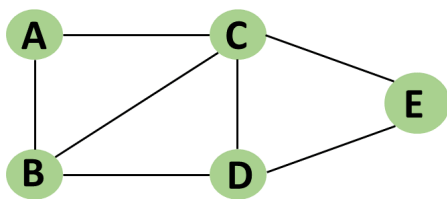
- Vertex E should be pushed to the top of the stack.



Step 6: Vertex E's nearby vertices, namely vertex C and D have been visited, pop vertex E from the stack.

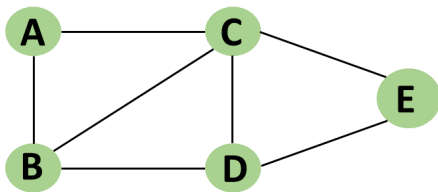


Step 7: Now that all of vertex D's nearby vertices, namely vertex B and C, have been visited, pop vertex D from the stack.



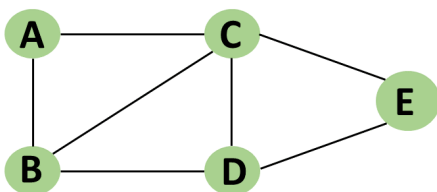
STACK

Step 8: Similarly, vertex C's adjacent vertices have already been visited; therefore, pop it from the stack.



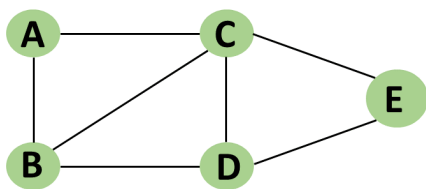
STACK

Step 9: There is no more unvisited adjacent vertex of b, thus pop it from the stack.



STACK

Step 10: All of the nearby vertices of Vertex A, B, and C, have already been visited, so pop vertex A from the stack as well.



STACK